

## JavaScript – L3 UPEC

### TP : Bataille navale – objets, références et closures

Dans ce TP, vous allez développer le moteur d'une petite bataille navale et le relier à une interface Web déjà fournie. Le travail porte principalement sur les notions étudiées en cours : objets, fonctions, références, fonctions d'ordre supérieur et closures.

L'objectif n'est pas seulement d'obtenir un programme qui fonctionne. On cherchera aussi à construire un programme dans lequel chaque partie possède une responsabilité claire.

**Fichiers fournis :** L'archive contient :

- `ship.js` : création et comportement des navires ;
- `board.js` : représentation de la grille et gestion des tirs ;
- `game.js` : état général d'une partie ;
- `appTest.js` : fichier réservé aux tests intermédiaires ;
- `view.js` et `app.js` : interface déjà fournie ;
- `index.html` et `style.css` : page de jeu déjà fournie.

Vous devez compléter uniquement `appTest.js`, `ship.js`, `board.js` et `game.js`.

**Exécution :** Aucune installation particulière n'est nécessaire. Décompressez l'archive puis ouvrez `index.html` dans un navigateur moderne. Rechargez la page après chaque modification.

**Méthodologie :** Pendant le TP, vous utiliserez régulièrement la console JavaScript du navigateur pour vérifier le comportement de vos fonctions avant de poursuivre.

Pour ouvrir la console :

- ouvrez `index.html` dans le navigateur ;
- ouvrez les outils de développement ;
- sélectionnez l'onglet **Console**.

La console permet notamment :

- d'afficher des valeurs avec `console.log(...)` ;
- de vérifier le résultat d'une expression ;
- de repérer les erreurs JavaScript éventuelles.

Pour les tests intermédiaires, utilisez le fichier `appTest.js` fourni. Ce fichier est chargé après les fichiers que vous devez compléter et peut donc appeler directement leurs fonctions.

Vous pouvez y écrire temporairement vos propres tests, par exemple :

```
1 const ship = createShip("Test", 2);
2
3 console.log(ship.getName());
4 console.log(ship.isSunk());
```

Après chaque modification de `appTest.js`, rechargez la page puis observez les résultats dans la console.

Le fichier `appTest.js` sert uniquement à vos tests intermédiaires : vous pouvez modifier son contenu librement pendant le TP.

**Discipline de conception :** Dans tout le TP, l'état interne d'un navire ne doit jamais être manipulé directement depuis l'extérieur. Le reste du programme doit communiquer avec un navire uniquement par les méthodes de l'objet retourné par `createShip`.

#### Exercice 1. Créer un navire

On souhaite écrire :

```
1 const cruiser = createShip("Croiseur", 3);
```

Chaque appel à `createShip` doit créer un nouveau navire indépendant.

**Choix de conception** Les informations suivantes appartiennent au navire : son nom, sa longueur et son nombre de touches reçues. Elles ne doivent pas être accessibles directement depuis l'extérieur.

Utilisez une **closure** pour conserver ces informations dans l'environnement de `createShip`. L'objet retourné donne uniquement accès aux opérations nécessaires au reste du programme. On obtient ainsi une véritable encapsulation.

L'objet retourné doit proposer l'interface suivante :

- `getName()` : retourne le nom du navire ;
- `getLength()` : retourne sa longueur ;
- `getHits()` : retourne son nombre de touches ;
- `hit()` : enregistre une touche ;
- `isSunk()` : indique si le navire est coulé.

Une touche supplémentaire ne doit pas pouvoir faire dépasser le nombre de touches au-delà de la longueur du navire.

Complétez `createShip` dans `ship.js`.

**Tests intermédiaires** Dans `appTest.js`, créez deux navires de longueurs différentes puis utilisez `console.log(...)` pour vérifier que :

- les deux objets sont distincts ;
- toucher un navire ne modifie pas l'autre ;
- `isSunk()` devient vrai au bon moment ;
- plusieurs appels supplémentaires à `hit()` ne rendent pas le nombre de touches incohérent.

Pour chaque propriété à vérifier, affichez explicitement dans la console les valeurs qui permettent de conclure.

## Exercice 2. Construire la grille

Une case de la grille est représentée par un objet de la forme :

```
1 { ship: null, shot: false }
```

Ici, `ship` ne contient pas les données du navire. Il contient soit `null`, soit une **référence vers un objet navire**.

**Étape 1 – Création** Complétez `createBoard(size)` dans `board.js`. La fonction doit retourner une grille carrée de taille `size`. Toutes les cases sont initialement vides et aucune n'a été jouée.

**Étape 2 – Placement d'un navire** Complétez :

```
1 placeShip(board, ship, row, col, direction)
```

`direction` vaut "horizontal" ou "vertical". La longueur du navire doit être obtenue par son interface publique.

**Point conceptuel important** Un navire de longueur 3 occupe trois cases, mais on ne crée pas trois navires. Les trois cases doivent contenir une référence vers **le même objet**.

Après avoir placé un navire, vérifiez expérimentalement que deux cases occupées par celui-ci contiennent bien la même référence avec l'opérateur `===`.

## Exercice 3. Effectuer un tir

Complétez :

```
1 shoot(board, row, col)
```

Cette fonction doit modifier l'état de la case visée et retourner un objet décrivant le résultat du tir.

Les quatre situations possibles sont :

- "miss" : la case est vide ;
- "hit" : un navire vient d'être touché mais reste à flot ;

- "sunk" : ce tir vient de couler le navire ;
  - "already-shot" : la case avait déjà été jouée.
- Le résultat doit respecter la forme générale suivante :

```
1 { status: ..., ship: ... }
```

La propriété `ship` vaut `null` lorsqu'aucun navire n'est concerné, et référence le navire concerné dans les autres cas où cela est utile.

**Discipline imposée** `shoot` ne doit connaître ni la représentation interne d'un navire, ni la manière dont celui-ci compte ses touches. Utilisez uniquement les méthodes exposées par le navire.

**Tests intermédiaires** Dans `appTest.js`, construisez une petite grille contenant au moins un navire, puis effectuez plusieurs tirs afin de produire successivement les cas :

- "miss";
- "hit";
- "sunk";
- "already-shot".

Affichez avec `console.log(...)` l'objet retourné par chaque appel à `shoot`.

#### Exercice 4. Créer une flotte

Complétez `createDefaultFleet()` dans `game.js`.

La flotte est composée des navires suivants :

| Navire       | Longueur | Ligne | Colonne | Direction   |
|--------------|----------|-------|---------|-------------|
| Croiseur     | 3        | 0     | 1       | horizontale |
| Frégate      | 3        | 2     | 6       | verticale   |
| Destroyer    | 2        | 5     | 1       | verticale   |
| Patrouilleur | 2        | 7     | 4       | horizontale |

La fonction doit créer des navires distincts et retourner les informations nécessaires à leur placement.

**Choix de conception** Les coordonnées ne font pas partie de l'état propre d'un navire : elles indiquent seulement où il est placé dans cette partie. On les conserve donc séparément de l'objet navire.

Cette séparation permettra par la suite de réutiliser un même type de navire avec d'autres placements.

#### Exercice 5. Encapsuler une partie

La fonction `createGame(size)` représente une partie complète.

Elle doit regrouper trois éléments d'état :

- la grille de jeu ;
- la flotte, c'est-à-dire le tableau contenant chaque navire une seule fois ;
- le nombre de tirs effectivement joués.

Ces informations ne doivent pas être accessibles directement depuis l'extérieur de la fonction.

Utilisez une nouvelle **closure** pour les conserver dans `createGame` et retournez un objet qui expose uniquement les opérations nécessaires au reste de l'application.

**Étape 1 – Construire l'état initial** Dans `createGame(size)` :

1. créez la grille avec `createBoard(size)` ;
2. récupérez les informations de placement avec `createDefaultFleet()` ;
3. construisez un tableau `fleet` contenant uniquement les navires ;
4. placez chaque navire sur la grille ;
5. initialisez le compteur de tirs.

**Pourquoi conserver une flotte séparément ?** La grille contient plusieurs références vers un même navire puisqu'un navire occupe plusieurs cases.

Le tableau `fleet`, au contraire, doit contenir chaque navire une seule fois. Il permet donc de raisonner facilement sur l'ensemble des navires : nombre de navires coulés, nombre total de touches, fin de partie, etc.

**Manipuler les tableaux** Pour construire et interroger la flotte, utilisez les méthodes de tableaux étudiées en cours.

- `map` permet de transformer chaque élément d'un tableau et de construire un nouveau tableau ;
- `forEach` permet d'appliquer une opération à chaque élément ;
- `filter` permet de conserver uniquement les éléments qui vérifient un prédicat ;
- `reduce` permet de combiner les éléments d'un tableau pour obtenir une seule valeur ;
- `every` permet de vérifier si tous les éléments satisfont un prédicat.

Dans ces appels, privilégiez les **fonctions fléchées** pour exprimer les petites fonctions utilisées comme paramètres.

Par exemple, si `values` est un tableau :

```

1 values.map(value => ...)
2 values.filter(value => ...)
3 values.every(value => ...)

```

Les expressions après `=>` sont volontairement laissées à compléter selon le besoin.

**Étape 2 – Effectuer un tir** Ajoutez une méthode :

```

1 fire(row, col)

```

Elle doit utiliser la fonction `shoot` déjà écrite.

Le compteur de tirs doit augmenter uniquement lorsqu'une nouvelle case est effectivement jouée.

Un second tir sur une case déjà jouée ne doit donc pas modifier ce compteur.

**Étape 3 – Exposer l'état sans permettre sa modification** L'objet retourné doit proposer :

- `getFleet()` : retourne une copie du tableau contenant la flotte. Une copie superficielle du tableau suffit ici. Vous pouvez utiliser l'opérateur de décomposition `...`
- `getBoard()` : retourne une copie de la grille et de ses cases. Attention, la grille est un tableau de tableaux contenant eux-mêmes des objets représentant les cases. Une simple copie du tableau extérieur ne suffit donc pas.

Par exemple, si l'on écrit :

```

1 const copy = [...board];

```

le tableau `copy` est bien différent de `board`, mais ses lignes référencent encore les mêmes tableaux. Il faut donc construire un nouveau tableau de lignes puis, pour chaque ligne, créer de nouveaux objets représentant les cases.

Les références vers les navires, en revanche, peuvent rester partagées : elles désignent volontairement les mêmes objets navires.

**Indication** Utilisez `map` à deux niveaux :

- un premier `map` pour reconstruire les lignes ;
- un second `map` pour créer une copie de chaque case.

Pour copier un objet case, vous pouvez utiliser l'opérateur de décomposition :

```

1 { ...cell }

```

Ces méthodes permettent de consulter l'état de la partie sans donner directement accès aux tableaux internes conservés dans la closure.

#### Étape 4 – Calculer les statistiques Ajoutez :

```
1 getStats()
```

Cette méthode doit retourner :

```
1 {
2   shots: ...,
3   hits: ...,
4   sunk: ...,
5   totalShips: ...
6 }
```

Le nombre de tirs est une information propre à la partie : il doit donc être conservé dans la closure.

Les autres valeurs peuvent être retrouvées à partir de la flotte :

- le nombre total de touches en interrogeant tous les navires ;
- le nombre de navires coulés en sélectionnant ceux pour lesquels `isSunk()` est vrai ;
- le nombre total de navires à partir de la taille du tableau.

Évitez de stocker une information qui peut être recalculée simplement à partir de l'état existant.

#### Étape 5 – Détecter la fin de partie Ajoutez :

```
1 isOver()
```

La partie est terminée lorsque tous les navires de la flotte sont coulés.

Utilisez une méthode de tableau adaptée pour exprimer directement cette propriété.

**Interface finale** L'objet retourné par `createGame(size)` doit finalement proposer :

- `fire(row, col)` : effectue un tir et retourne son résultat ;
- `getBoard()` : retourne une copie de la grille et de ses cases ;
- `getFleet()` : retourne une copie du tableau contenant la flotte ;
- `getStats()` : retourne les statistiques courantes ;
- `isOver()` : indique si toute la flotte est coulée.

**Tests intermédiaires** Dans `appTest.js`, créez une partie avec `createGame()` puis utilisez uniquement son interface publique pour vérifier :

- l'état initial de la partie ;
- l'évolution du nombre de tirs ;
- qu'un tir répété ne modifie pas ce compteur ;
- l'évolution du nombre de touches ;
- le passage d'un navire à l'état coulé ;
- le comportement de `getFleet()` et `getBoard()`.

Vérifiez notamment avec `===` que les tableaux retournés sont distincts des tableaux internes, tout en observant quelles références vers les navires restent partagées.

### Validation finale et lecture de l'architecture

Lorsque `ship.js`, `board.js` et `game.js` sont terminés, la page doit permettre de jouer une partie complète.

Vérifiez les situations suivantes :

1. un tir dans l'eau ;
2. une touche ;
3. un navire coulé ;
4. un second clic sur une case déjà jouée ;
5. la victoire lorsque tous les navires sont coulés ;
6. le démarrage d'une nouvelle partie.